

## Code-Layout, Readability and Reusability

|                      |              |
|----------------------|--------------|
| <b>Date</b>          | 28 June 2026 |
| <b>Team ID</b>       |              |
| <b>Project Name</b>  | Smart Lender |
| <b>Maximum Marks</b> | 5 Marks      |

### Code-Layout, Readability and Reusability:

The Smart Lender project is developed using a clean and organized code structure. It uses reusable modules for preprocessing, model training, and prediction. Proper variable names, comments, and error handling improve readability and maintenance. Overall, the project follows good coding practices and provides an efficient and user-friendly system.

### Code Layout Checklist:

| S.No | Code Quality Parameter    | Description                                   | Followed (Yes / No / Partial) | Remarks                                                                              |
|------|---------------------------|-----------------------------------------------|-------------------------------|--------------------------------------------------------------------------------------|
| 1    | Consistent Indentation    | Uniform spacing/tabs used throughout the code | Yes                           | Python code follows proper indentation and formatting standards.                     |
| 2    | Proper File Structure     | Files and folders are logically organized     | Yes                           | Project is organised into models, templates, datasets, statis and python files.      |
| 3    | Meaningful Variable Names | Variables reflect their purpose clearly       | Yes                           | Variables like loan_amount, credit_hists, and applicant_income are self-explanatory. |
| 4    | Function / Method Names   | Functions are descriptively named             | Yes                           | Functions such as preprocess (), predict () clearly indicate their purpose.          |
| 5    | Code Comments             | Inline and block comments explain logic       | Yes                           | Comments are provided in preprocessing and model-related files.                      |
| 6    | Modular Design            | Code is split into reusable functions/modules | Yes                           | Separate modules for preprocessing, model training, and Flask application.           |
| 7    | No Redundant Code         | Duplicate or unused code is removed           | Partial                       | Some repetitive form-handling code can be further optimized.                         |

### Reusable Components / Modules:

| S.No | Component / Module Name                   | Language / Technology        | Where Reused                             | Reusability Level |
|------|-------------------------------------------|------------------------------|------------------------------------------|-------------------|
| 1    | Data Preprocessing Module (preprocess.py) | Python, Pandas, Scikit-learn | Training and model development           | High              |
| 2    | Standard Scaler (scaler.pkl)              | Scikit-learn                 | Used during training and prediction      | High              |
| 3    | Trained Model (best_model.pkl)            | Scikit-learn                 | Used by Flask application for prediction | High              |
| 4    | Flask Prediction Function (predict())     | Flask, Python                | Handles all prediction requests          | Medium            |
| 5    | HTML & CSS Templates                      | HTML, CSS                    | Reused across multiple web pages         | Medium            |

### Overall Code Quality Assessment:

| Aspect                   | Rating (1-5) | Comments                                                                                                   |
|--------------------------|--------------|------------------------------------------------------------------------------------------------------------|
| Code Layout & Structure  | 5            | Well-organized project structure with separate folders and modules.                                        |
| Readability              | 5            | Clear variable names and easy-to-understand code.                                                          |
| Reusability              | 4            | Several reusable modules; minor improvements can further increase reusability.                             |
| Documentation / Comments | 4            | Good comments are present, but more documentation can be added.                                            |
| Overall Score            | 4.5/5        | The project follows good coding practices and demonstrates a well-structured machine learning application. |